Apache Spark & PySpark

Лекция 5

Зачем нам Spark

Spark нужен, когда данные или вычисления уже перестают быть задачей одного ноутбука.

Он дает единый движок для ETL, SQL, Streaming и аналитики на кластере.

Распределенные вычисления

Нужны когда данных много

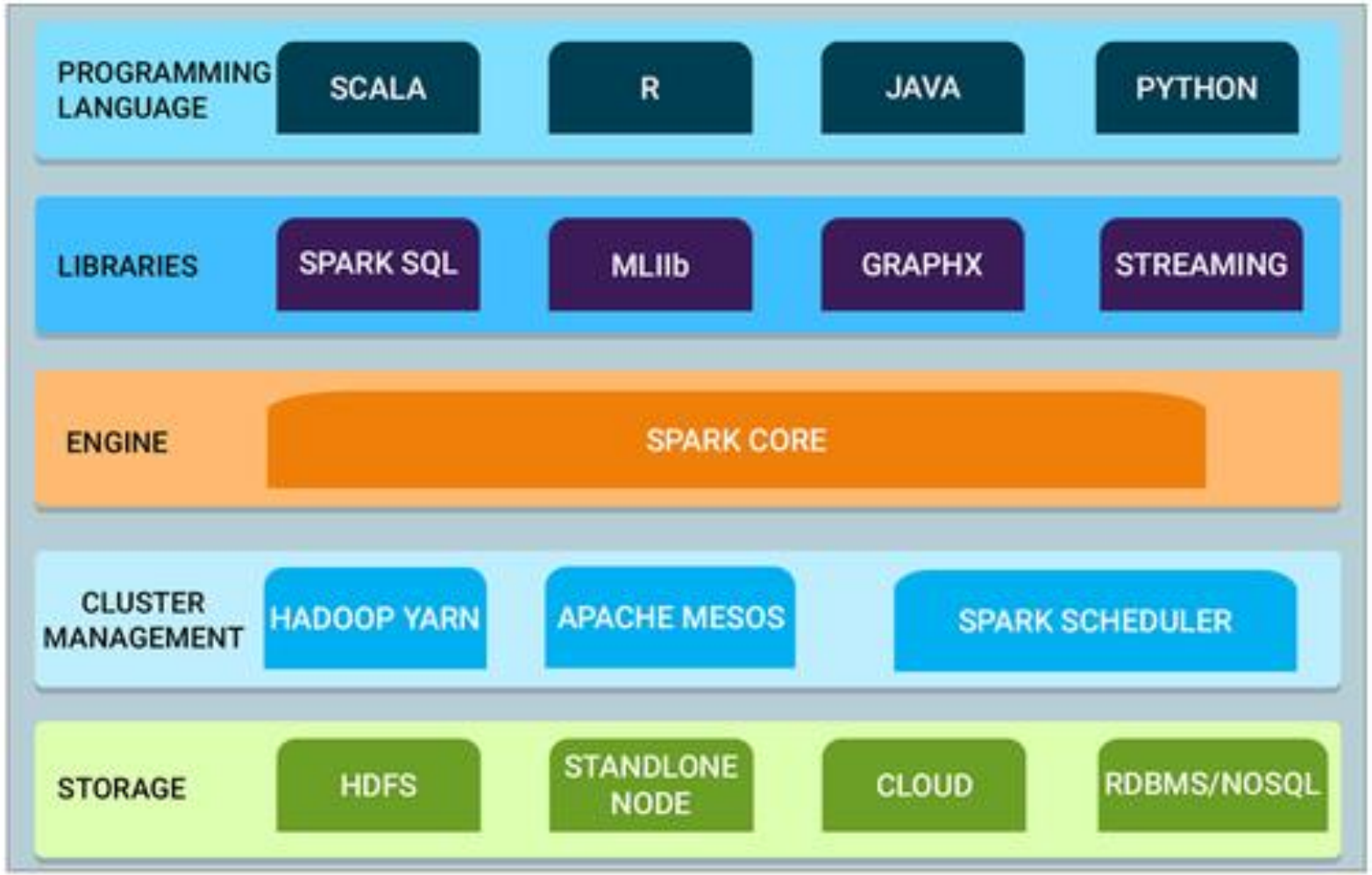
Знаем про HDFS и MapReduce

Основная проблема Hadoop MapReduce:

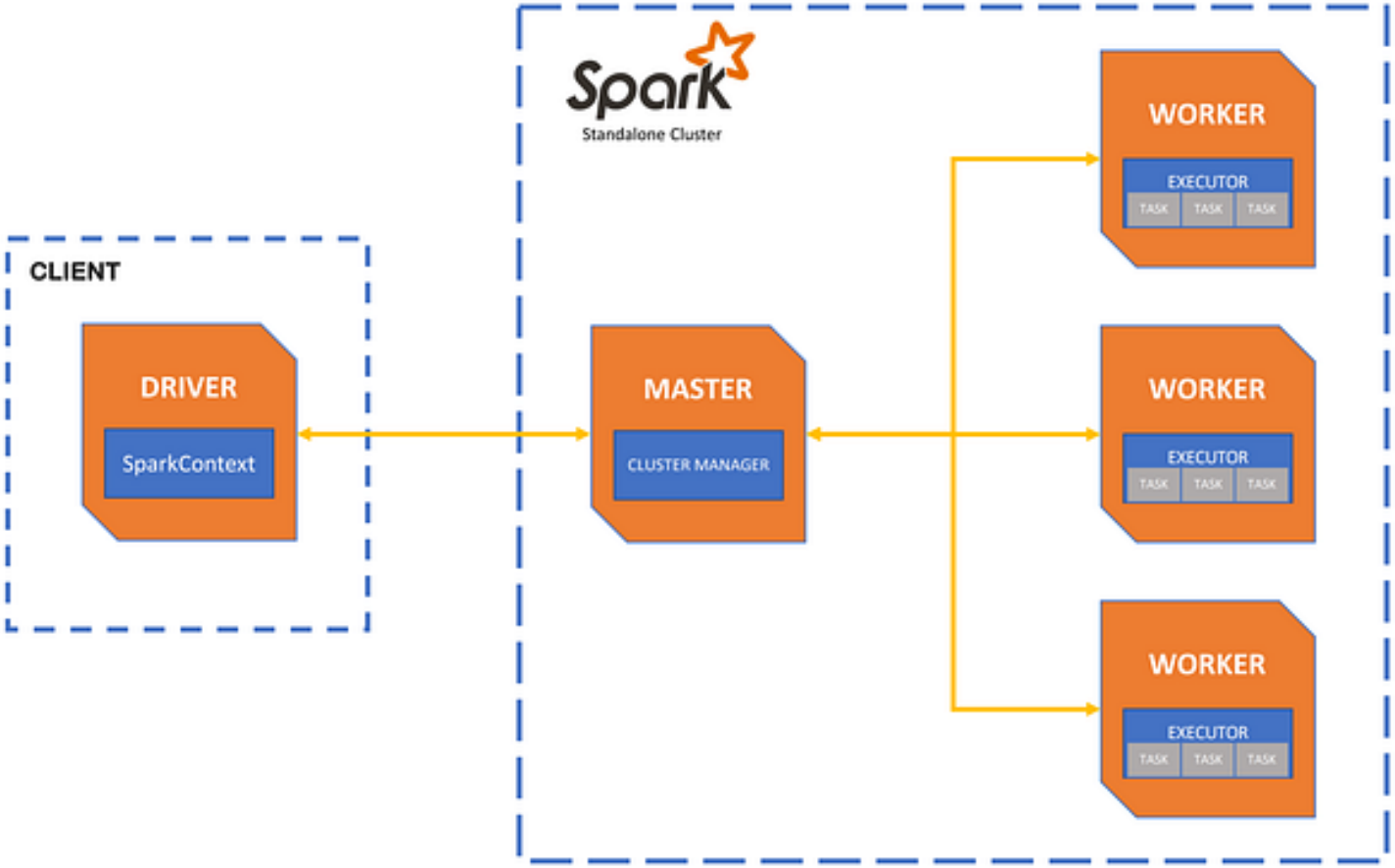
Читает и пишет данные на диск после каждого шага

Spark минимизирует R/W операции, за счет чего можно добиться ускорения до **100x раз**

Компоненты Spark



Компоненты кластера Spark



SparkSession и источники данных

Точка входа в PySpark — SparkSession.

Читаем CSV, JSON, Parquet, ORC, Hive-таблицы, JDBC-источники.

В production лучше задавать schema явно.

inferSchema удобен для демо, но может вести к нестабильным пайплайнам.

DataFrame API: с чего начинаем

Базовые операции: select, selectExpr, withColumn, filter, orderBy, limit.

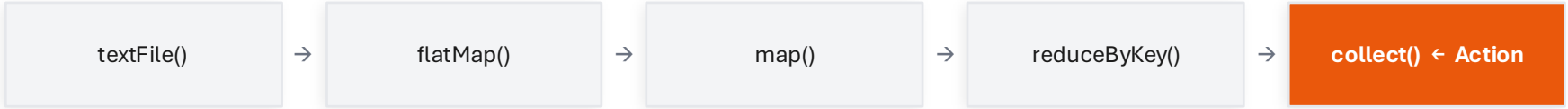
Колонка в Spark — это отдельный объект, а не обычный Python-объект.

Для большинства операций есть функции в pyspark.sql.functions.

DAG и ленивые вычисления

Spark не выполняет трансформации немедленно.

Он строит граф операций (DAG) и запускает его только при вызове действия.



Трансформации (Lazy)

Не выполняются сразу.
Пример: map, filter, flatMap, join

Spark только добавляет их в DAG.

Действия (Eager)

Запускают выполнение DAG.
Пример: collect, count, save, show

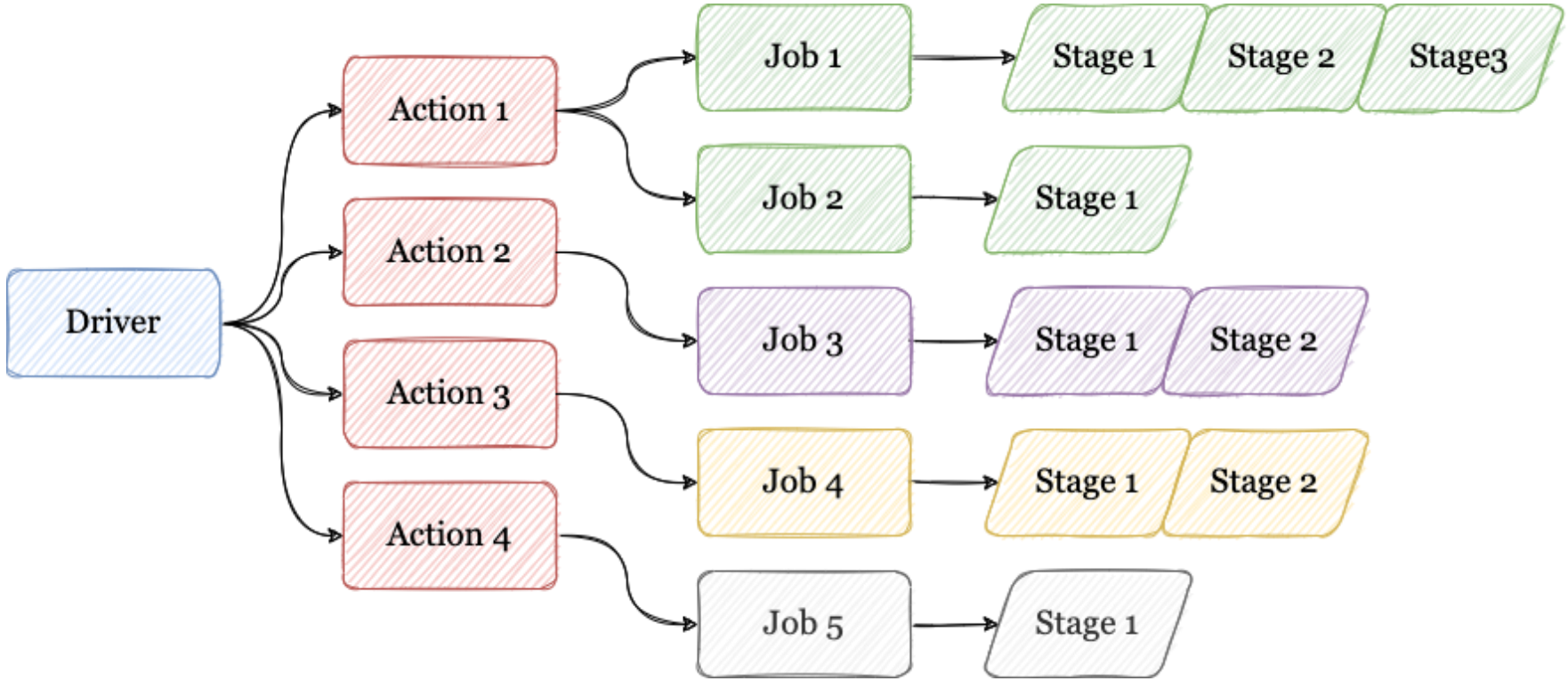
Только тогда данные реально обрабатываются.

Jobs, stages и tasks

Job делится на **Stages** по границам *shuffle*

Stage состоит из **Tasks** по числу *partitions*

Именно **Task** является базовой единицей параллелизма.



Узкие и широкие трансформации

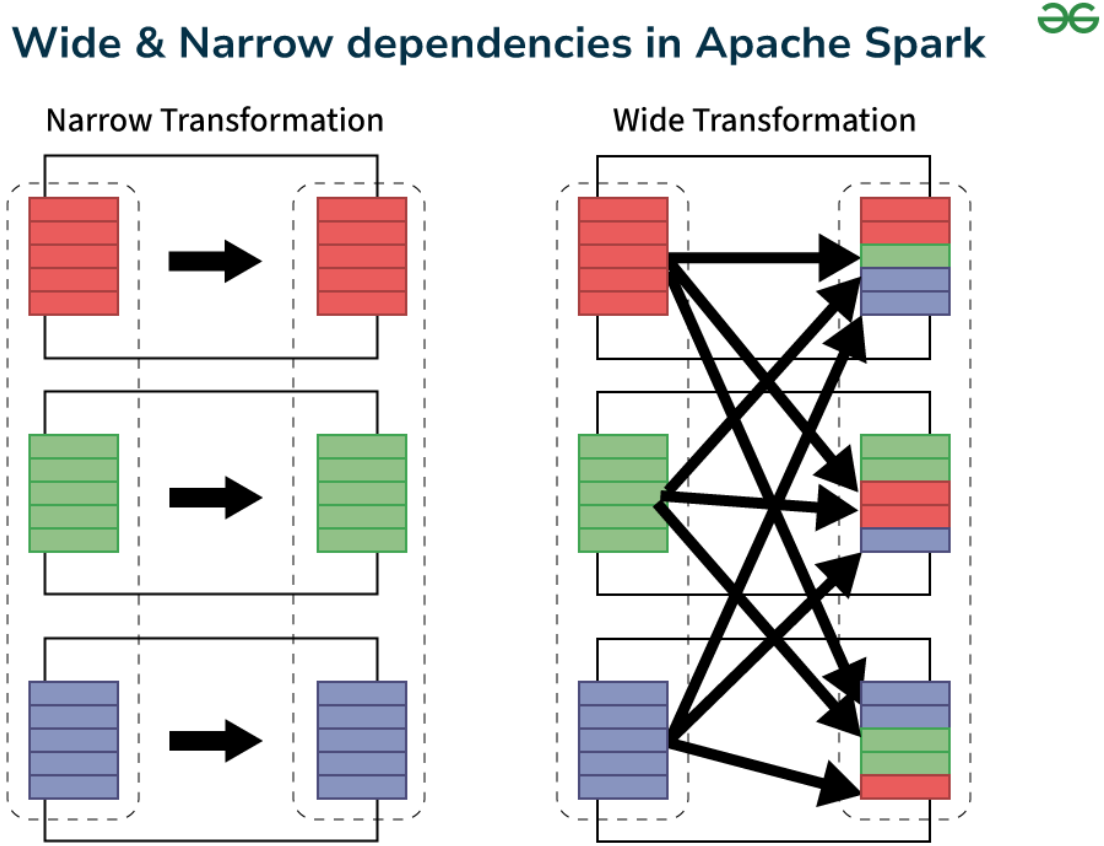
Узкие

Выходная партиция зависит **только** от одной входной партиции

Примеры:

- Map
- Filter
- Select
- withColumn

Дешевые трансформации



Широкие

Данные нужно перегруппировать между partitions

Примеры:

- groupBy
- Join
- Distinct
- orderBy

Дорогие трансформации

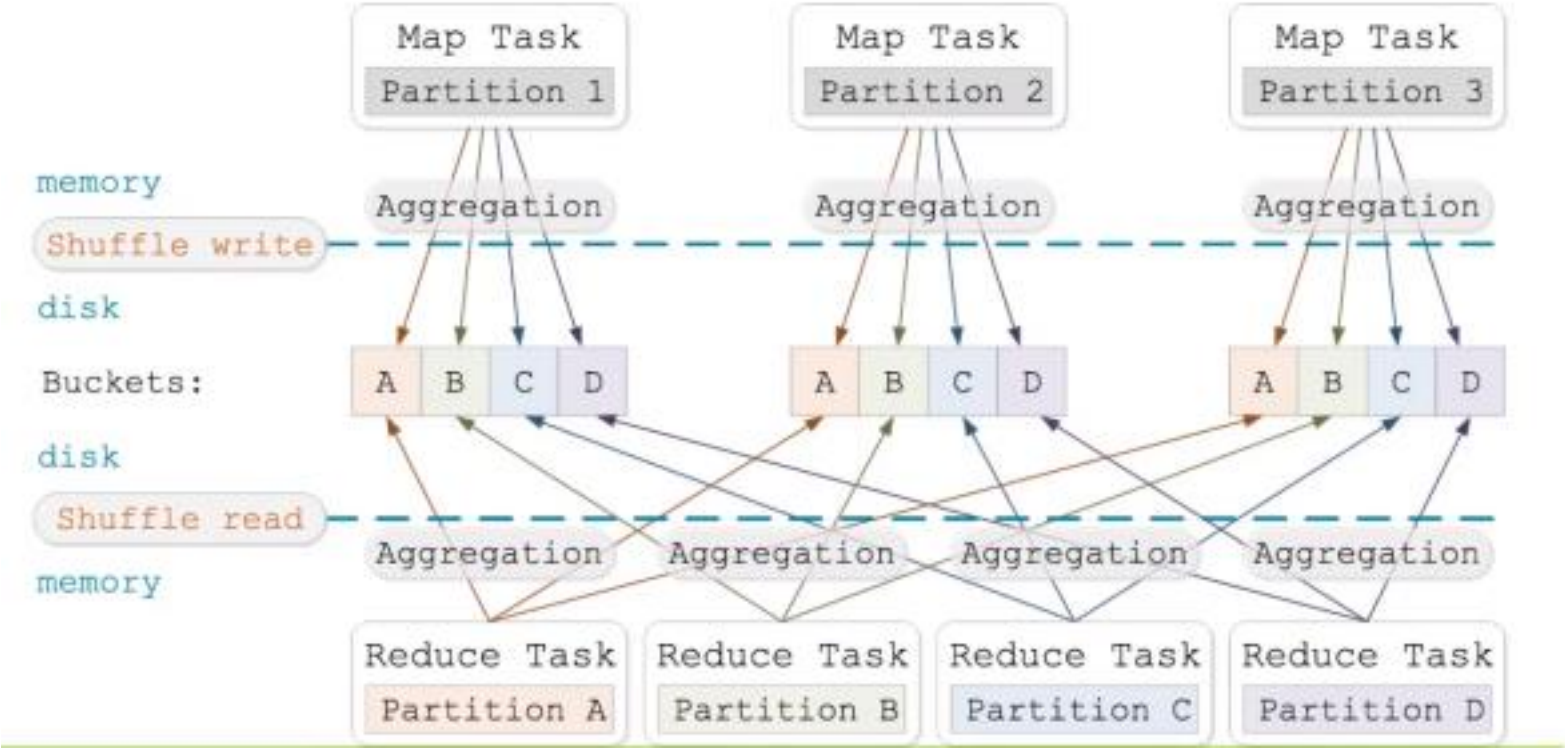
Shuffle и partitioning

Shuffle — это пересылка данных между executors

Он дорог по сети, диску, сериализации и сортировке.

Слишком мало partitions — мало параллелизма.

Слишком много partitions — overhead и много мелких файлов.



Cache, Persist, Checkpoint

Cache/Persist полезен, когда один DataFrame используется много раз

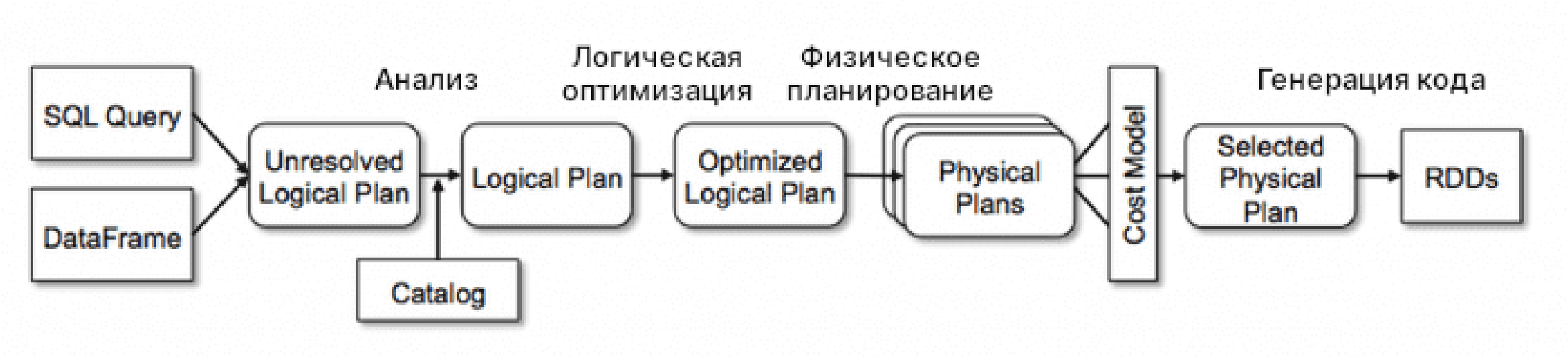
Cache расходует память и не должен включаться "на всякий случай”

Помогает «зафиксировать» момент и не пересчитывать все заново

Можно так же пользовать checkpoint

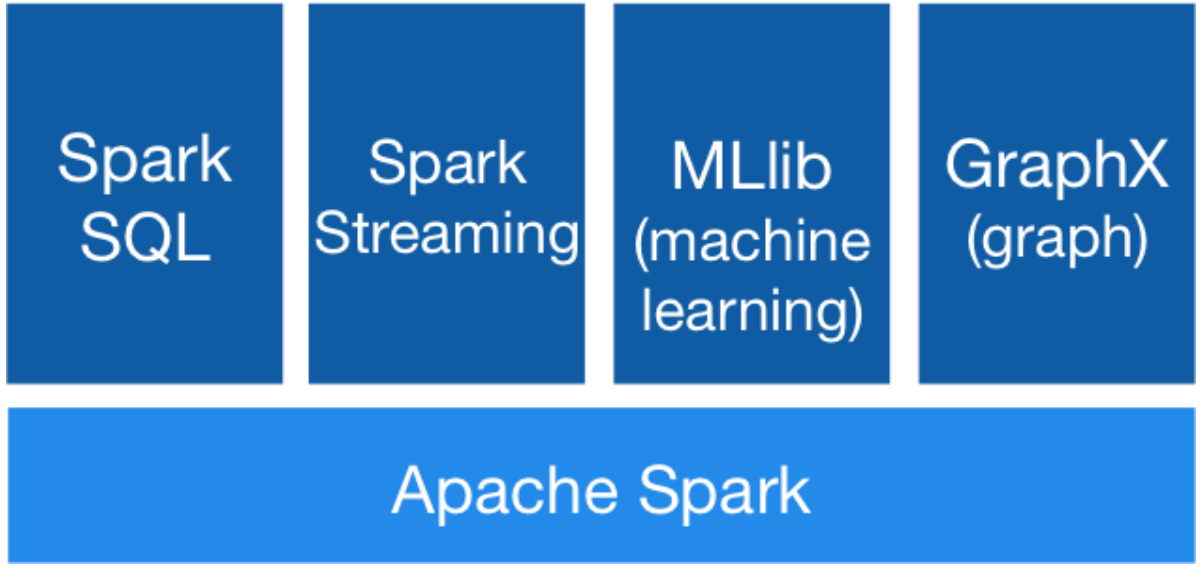
Catalyst

Catalyst оптимизирует запросы в Spark

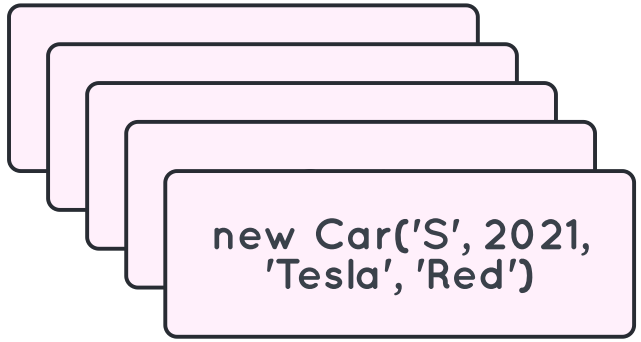


Абстракции Spark

- RDD — низкоуровневая абстракция распределённых коллекций.
- DataFrame — основная табличная абстракция в современном Spark.
- Dataset особенно удобен в Scala/Java, но в PySpark почти всегда стартуем с DataFrame API.



RDD



Dataframe

model	year	brand	color
S	2021	Tesla	Red
X	2022	Tesla	White
S	2023	Honda	White
Y	2024	Kia	Red
Z	2020	Ford	Blue

Dataset

new Car	S	2021	Tesla	Red
new Car	X	2022	Tesla	White
new Car	S	2023	Honda	White
new Car	Y	2024	Kia	Red
new Car	Z	2020	Ford	Blue

Когда Spark — правильный инструмент

Spark оправдан, когда объём данных, ширина таблиц или время расчёта уже выходят за рамки локальной аналитики.

- Если задача помещается в pandas или хорошо решается прямо в СУБД, Spark может быть лишней сложностью.
- У Spark есть цена запуска, shuffle и распределённой отладки.

Группировки и агрегации

groupBy + agg — основной способ считать бизнес-метрики.

Типичные функции: count, sum, avg, min, max, countDistinct.

После groupBy часто возникает shuffle

Если можно уменьшить данные до join — стоит агрегировать раньше.

https://medium.com/@nambastha07/spark-optimisation-every-groupby-is-a-shuffle-and-how-to-reduce-it-9313b1d7a4f5

Join

LEFT JOIN

Everything on the left
+
anything on the right that matches

```
SELECT *
FROM TABLE_1
LEFT JOIN TABLE_2
ON TABLE_1.KEY = TABLE_2.KEY
```

ANTI LEFT JOIN

Everything on the left
that is NOT on the right

```
SELECT *
FROM TABLE_1
LEFT JOIN TABLE_2
ON TABLE_1.KEY = TABLE_2.KEY
WHERE TABLE_2.KEY IS NULL
```

RIGHT JOIN

Everything on the right
+
anything on the left that matches

```
SELECT *
FROM TABLE_1
RIGHT JOIN TABLE_2
ON TABLE_1.KEY = TABLE_2.KEY
```

ANTI RIGHT JOIN

Everything on the right
that is NOT on the left

```
SELECT *
FROM TABLE_1
RIGHT JOIN TABLE_2
ON TABLE_1.KEY = TABLE_2.KEY
WHERE TABLE_1.KEY IS NULL
```

OUTER JOIN

Everything on the right
+
Everything on the left

```
SELECT *
FROM TABLE_1
OUTER JOIN TABLE_2
ON TABLE_1.KEY = TABLE_2.KEY
```

ANTI OUTER JOIN

Everything on the left and right
that is unique to each side

```
SELECT *
FROM TABLE_1
OUTER JOIN TABLE_2
ON TABLE_1.KEY = TABLE_2.KEY
WHERE TABLE_1.KEY IS NULL
OR TABLE_2.KEY IS NULL
```

INNER JOIN

Only the things that match on the left AND the right

```
SELECT *
FROM TABLE_1
INNER JOIN TABLE_2
ON TABLE_1.KEY = TABLE_2.KEY
```

CROSS JOIN

All combination of rows from the right and the left (cartesean product)

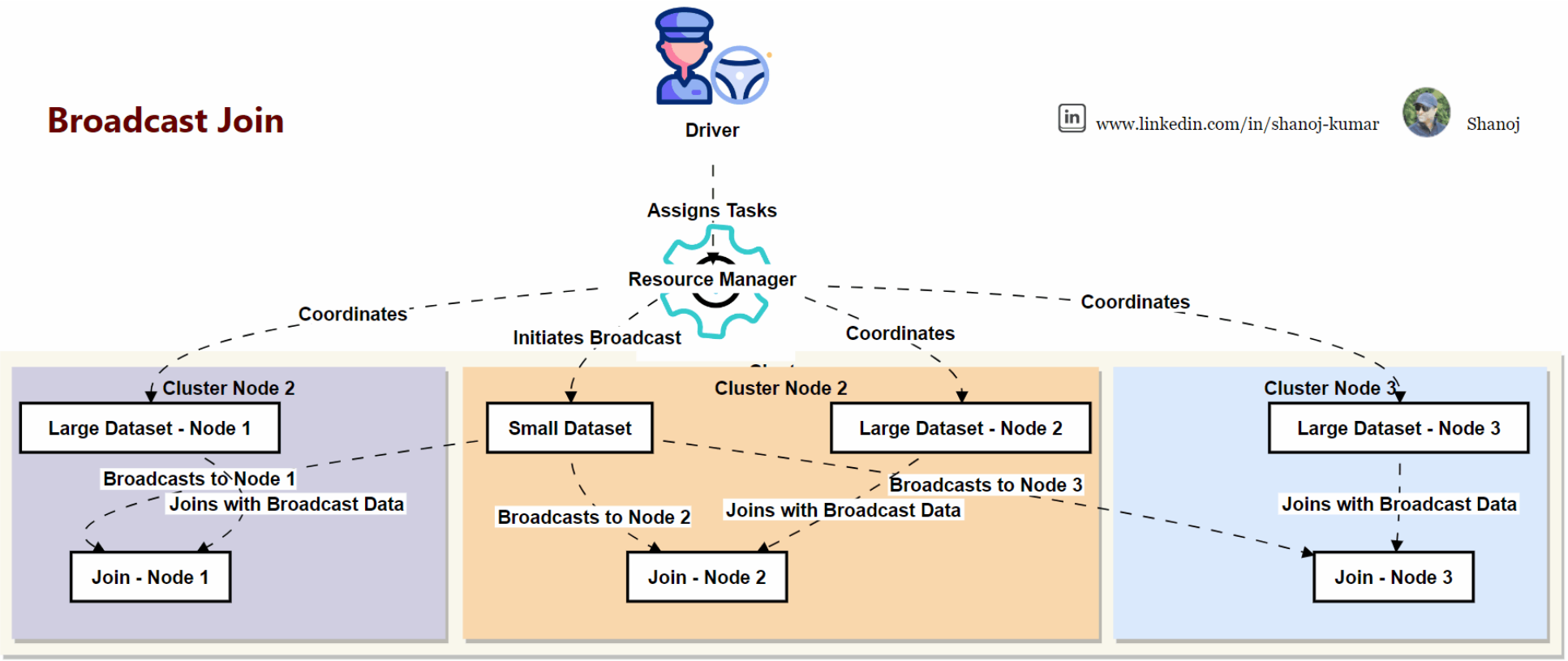
```
SELECT *
FROM TABLE_1
CROSS JOIN TABLE_2
```

Join Strategy

	Cartesian Join	Broadcast Hash Join	Sort merge Join	Shuffle Hash Join	Broadcast Nested Loop Join
Inner-Join	✓	✓	✓	✓	✓
Left-Join	✗	✓	✓	✓	✓
Right-Join	✗	✓	✓	✓	✓
Cross-Join	✗	✓	✓	✓	✓
Left-Semi_Join	✗	✓	✓	✓	✓
Left-Anti-Semi-Join	✗	✓	✓	✓	✓
Outer-Join	✗	✗	✓	✓	✓
Support Equi Join	✓	✓	✓	✓	✓
Support Non Equi Join	✓	✗	✗	✗	✓
Require Sortable Join key	✗	✗	✓	✗	✗

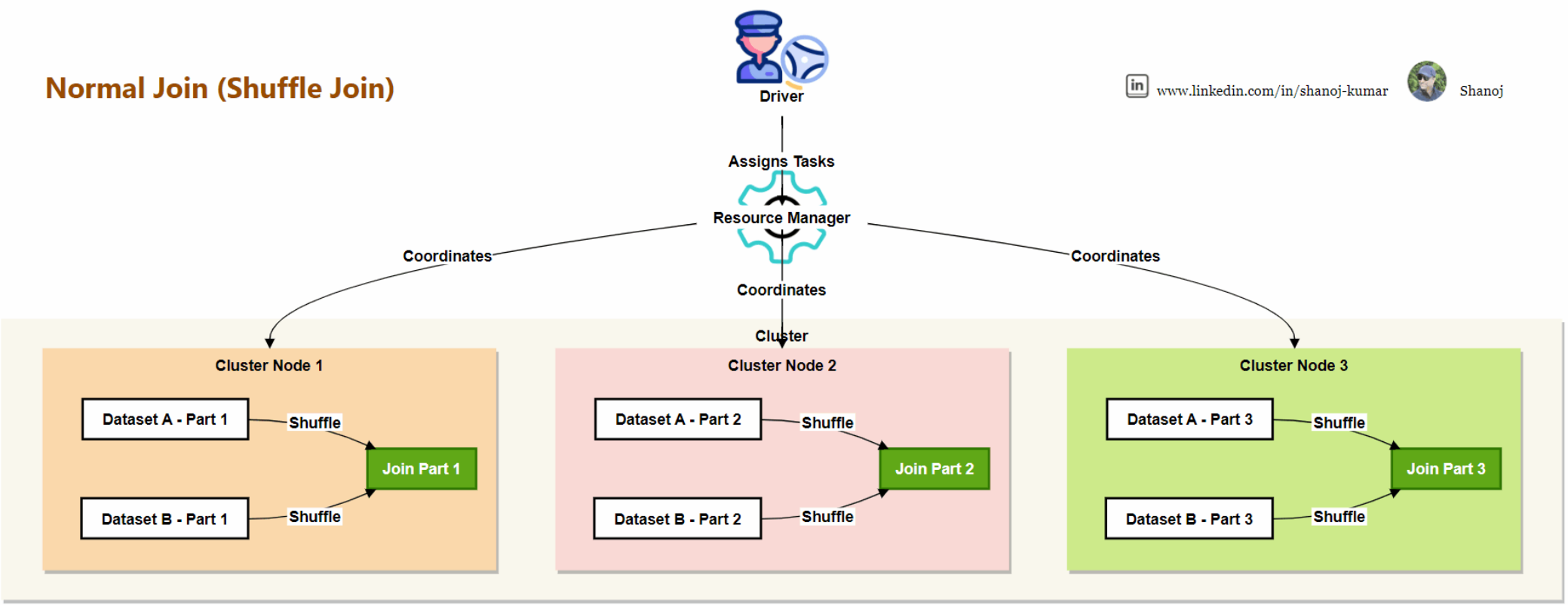
Broadcast Hash Join

Маленькая таблица **рассылается (broadcast)** на все executors, где джоинится с большой таблицей.



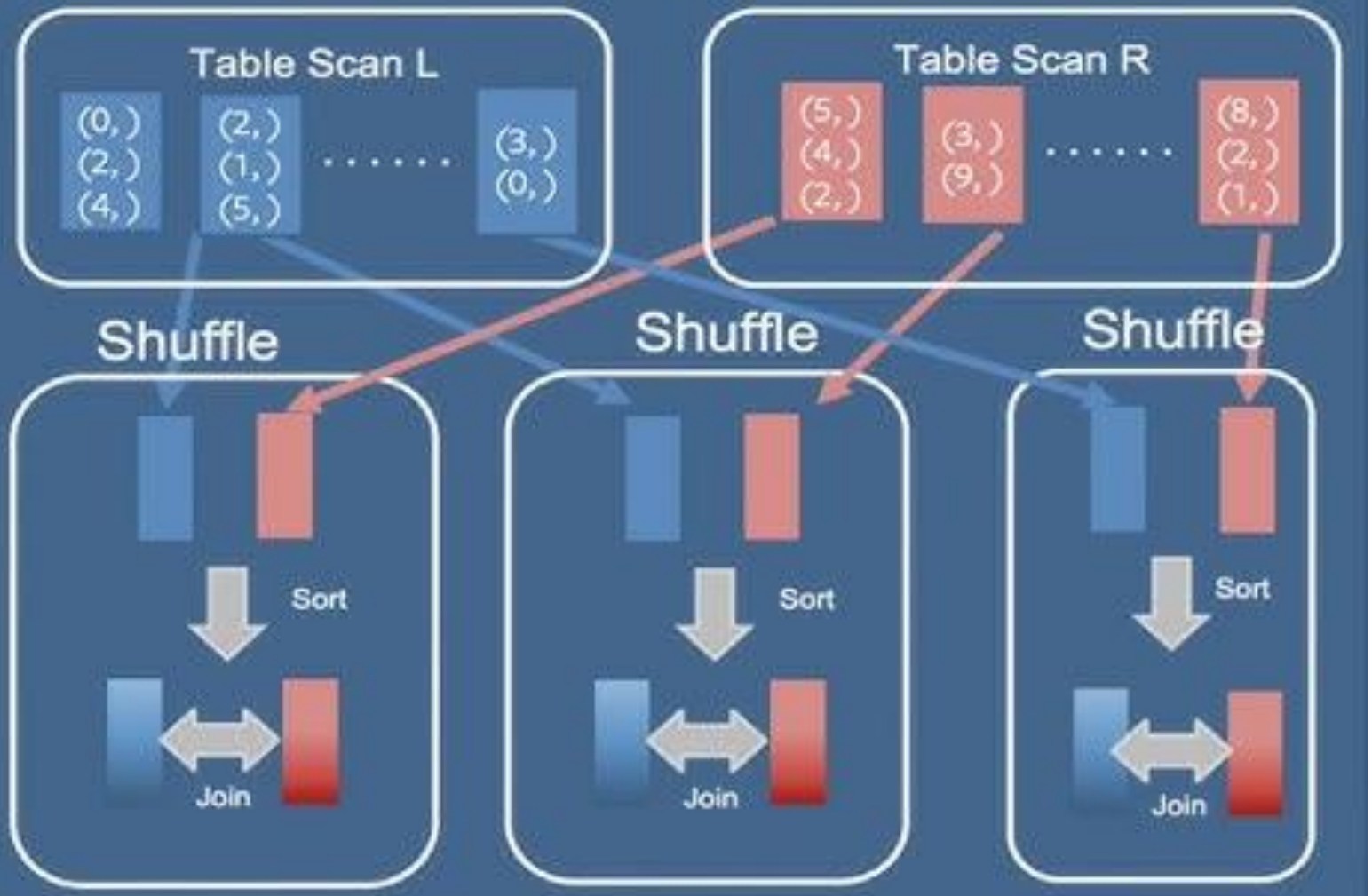
Shuffle Hash Join

Обе таблицы перемешиваются (shuffle) по ключам для джоина

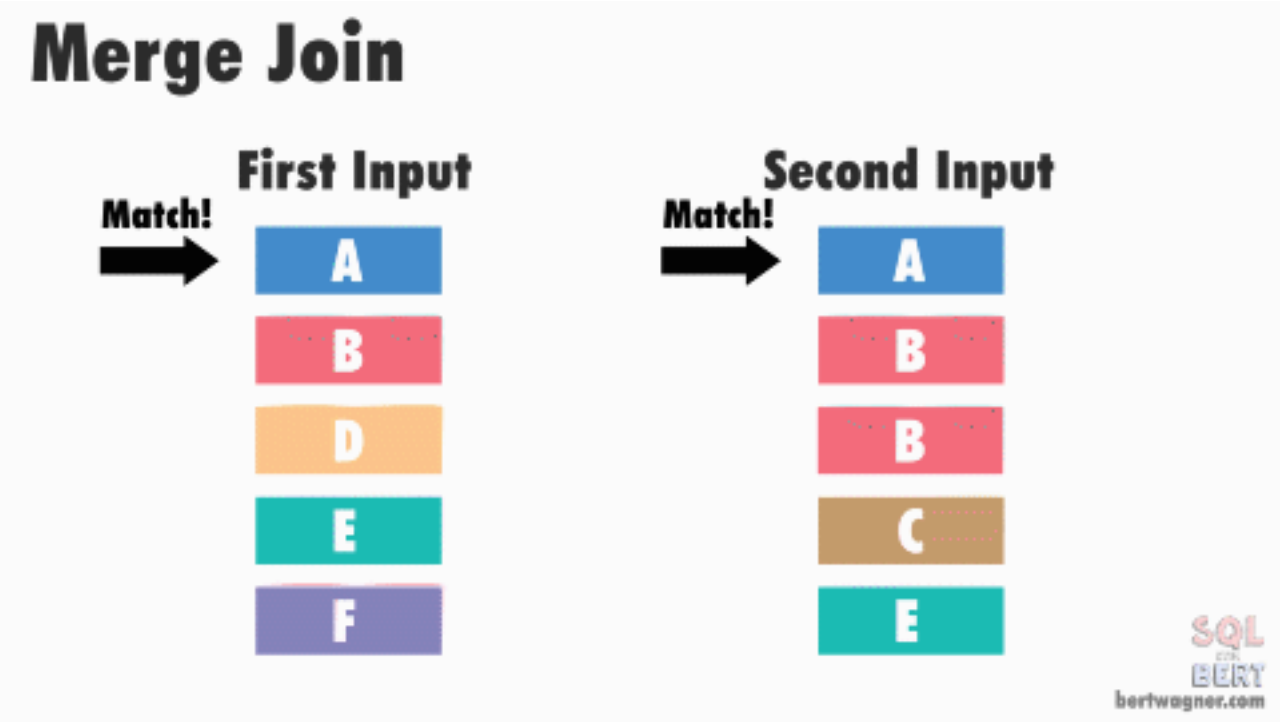


Sort merge join

- Shuffle both tables
- Sort both tables
- Join by buffer one, stream the bigger one



Sort Merge Join часть 2

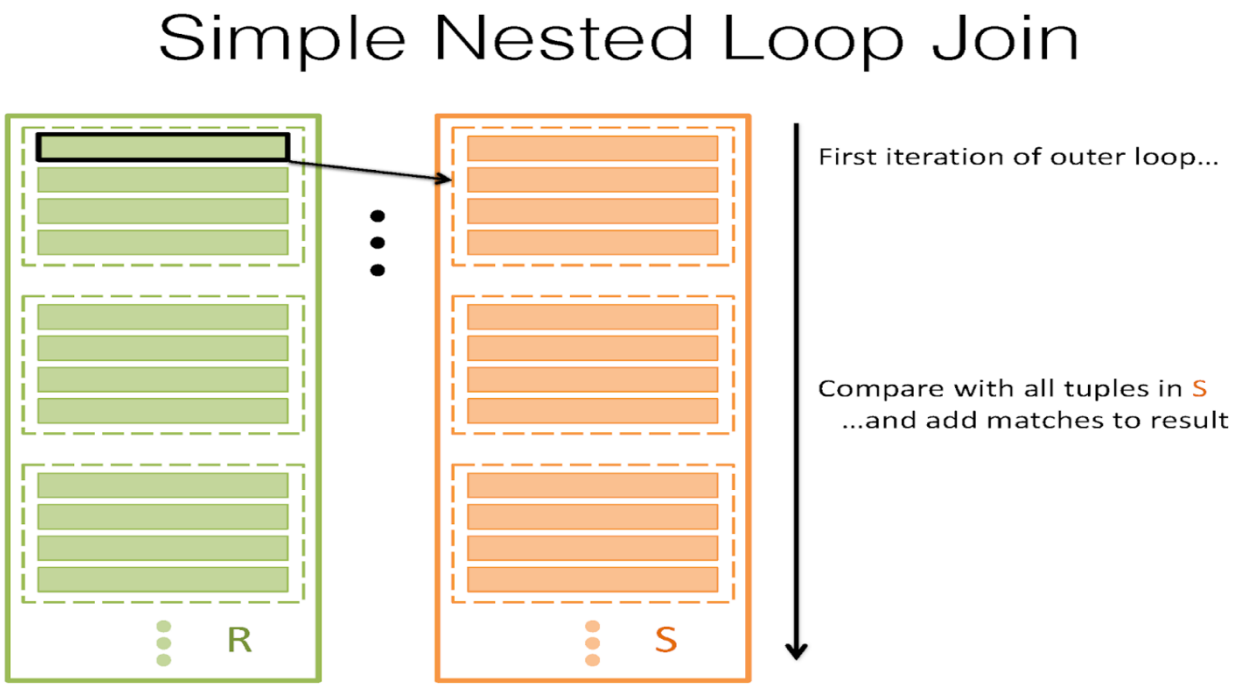


Broadcast Nested Loop Join

То же, что и в broadcast – данные маленькой таблицы рассылаются на все воркеры

Чуточку другой Join

Использует Nested Loop



Оконные функции

partitionBy задаёт группу, orderBy задаёт порядок внутри группы.

row_number, rank, dense_rank — топ-N и ранжирование.

sum over window — накопительные суммы

Parquet, partitioning таблиц

Parquet — колоночный формат, удобный для аналитики и ETL.

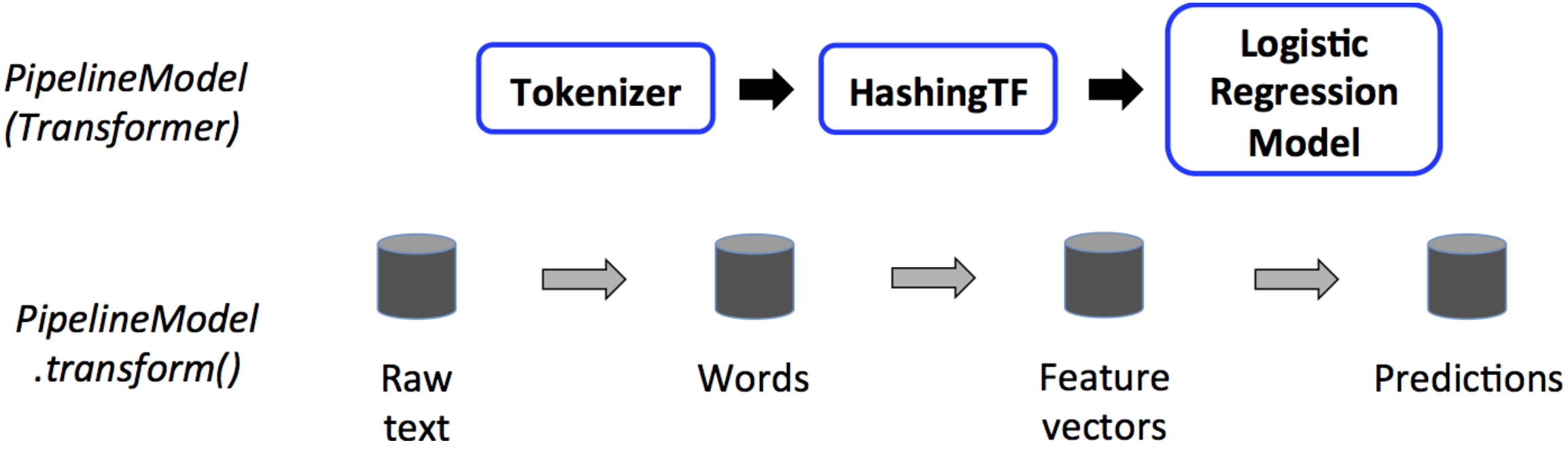
partitionBy полезен, если фильтрация часто идёт по дате, городу или статусу.

Слишком много мелких файлов замедляет чтение и усложняет pipeline.

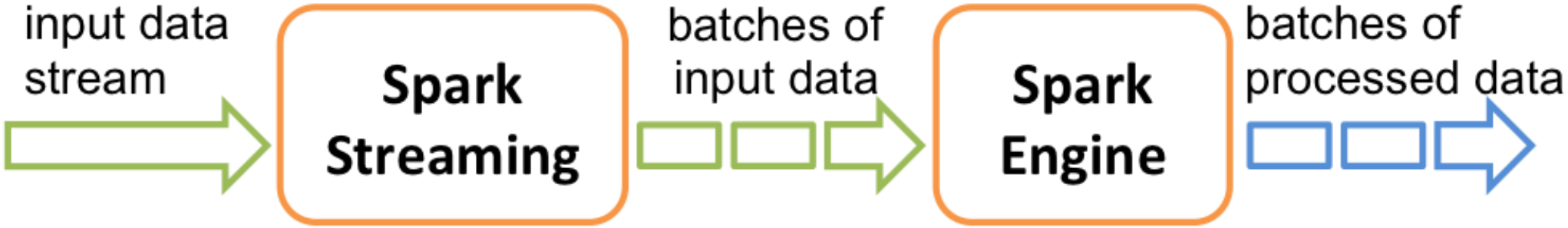
Подводные камни: где пользователи чаще всего ошибаются

- collect() и toPandas() на больших данных быстро ломают ноутбук или драйвер.
- Python UDF почти всегда медленнее встроенных Spark-функций.
- inferSchema, wide join и orderBy без необходимости делают pipeline тяжелее и хрупче.
- Skew по ключам, null-ы, дубликаты нужно проверять отдельно.

ML Lib


PipelineModel
(Transformer)
Tokenizer
HashingTF
Logistic
Regression
Model
PipelineModel
.transform()
Raw
text
Words
Feature
vectors
Predictions

Spark Streaming



Домашнее задание